Parallel Computing Paradigms, etc

Recap: (Software) Models in Parallel Programming

- Shared Memory
- Threads
- Message Passing
- Data Parallel
- Hybrid(Ex: Posix + MPI)
- SPMD (Single Program Multiple Data)
- MPMD (Multiple Program Multiple Data)

Although not apparent, such models are not specific to the particular type of machine or memory architecture. For example it is possible to emulate Message Passing on Shared Memory architectures and vice versa (although in a very "expensive‟ manner)

Shared-Address-Space Platforms (Shared Memory)

- Part (or all) the memory is directly accessible by all processors

- Processors interact by modifying data stored in this "space" shared

- If the time taken by a processor to access any memory word of the system (local or global) is identical, the platform is classified as **uniform memory access (UMA)**, otherwise, as the machine **non-uniform memory access (NUMA)**

NUMA and UMA Shared-Address-Space Platforms

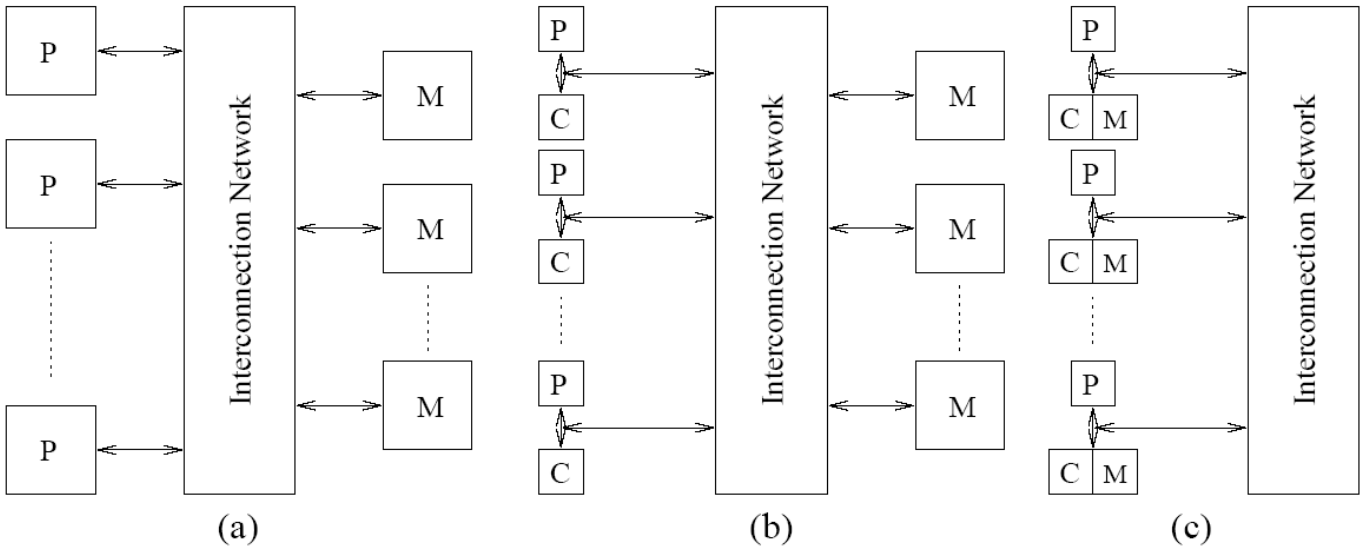


Figure 2.5 Typical shared-address-space architectures: (a) Uniform-memory-access shared-address-space computer; (b) Uniform-memory-access shared-address-space computer with caches and memories; (c) Non-uniform-memory-access shared-address-space computer with local memory only.

Shared-Address-Space Classes

Uniform Memory Access (UMA):

- Mainly represented by symmetric multiprocessor (SMP) machines
- Identical processors
- Identical memory access times
- Sometimes called CC-UMA - Cache Coherent UMA. Cache coherent means if one processor updates a location in shared memory, other processors are aware of the change. The **cache coherency** is done at the hardware level

Non-Uniform Memory Access (NUMA):

- Implemented by physically connecting two or more SMP
- One SMP can directly access the memory of the SMP
- Not all processors have equal access time to memory
- Access times slower through the links
- If the cache coherency is maintained, it is called CC-NUMA - Cache Coherent NUMA

Pros & Cons (Shared-Address-Space)

Advantages:

The space addressing allows a user-friendly type programming

The sharing of data between tasks is fast and uniform due to the "proximity" of the memory to the CPU

Disadvantages:

The main drawback is the lack of scalability between memory and CPUs. Adding more CPUs, there is an “geometrical” increase traffic on the shared bus memory CPU, and cache coherent systems, a geometric increase in traffic cache / memory

Programmer's responsibility to ensure that **synchronization** constructs a correct access to the global memory

Cost: it is **difficult and expensive** to design and produce shared memory machines with a greater number of processors ... (max. 64-128 proc)

NUMA and UMA Shared-Address-Space Platforms

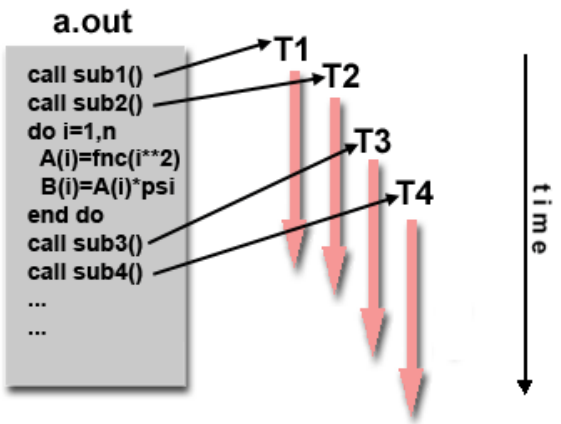
- The distinction between UMA and NUMA platforms is important from the algorithmic design point of view. NUMA machines require “**locality**" in the algorithms to achieve good performance
- However, read-write operations to shared data must be **coordinated**
- **Caches of these machines require coordinated access to multiple copies**. This leads to the problem of **cache coherence (later...)**

Methods for Multiprocessor Programming (Shared Memory)

- **Threads** (Pthreads, Python, Java, etc), where the programmer decomposes the program into individual parallel sequences, each being a "thread", and able to access variables declared outside this thread
- A (standard) sequential language, with preprocessor compilation directives, suitable for the specification of shared variables and the specification of parallelism. ES: **OpenMP** (industry standard)
- A (standard) compiler language with libraries (eg, methods, etc) for the declaration and access to shared variables
- A parallel programming language with parallelism syntax, in which the compiler creates the appropriate execution code for each processor (es: **Julia**)
- A sequential programming language, combined with a "parallel" ad hoc compiler for the conversion into parallel code

Threads Model

In the threads model, a single process can have several concurrent "paths"

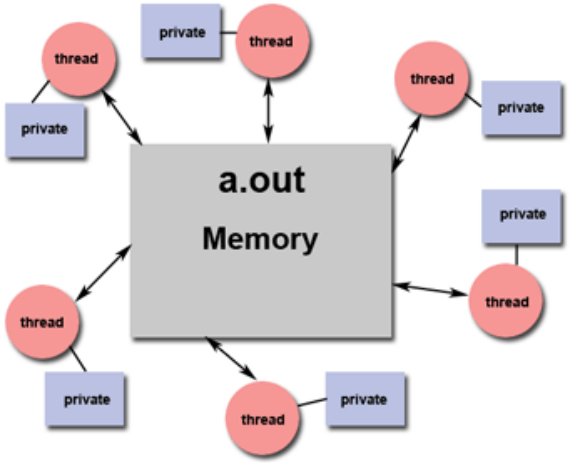


- The main program `a.out` is scheduled to be run by the native operating system. `a.out` is loaded and acquires the system and user resources needed to be executed

- `a.out` performs work sequentially, then creates a number of tasks (ie, threads) that can be scheduled and executed by the operating system concurrently

- Each thread has local data, but also shares the entire resources of `a.out`. This saves overhead generated by the replication of program resources for each thread. Each thread also benefits from the global view of shared memory of `a.out`.

- Threads are commonly associated with shared memory architectures and operating systems



Threads Implementions

From a programming point of view, the main thread implementations include:

- A library of subroutines that are called from within the parallel code
- A set of compiler directives embedded in both parallel and sequential code

POSIX Threads

Based on libraries; requires parallel coding

Specified by the IEEE POSIX 1003.1c (1995).

Only for the C language

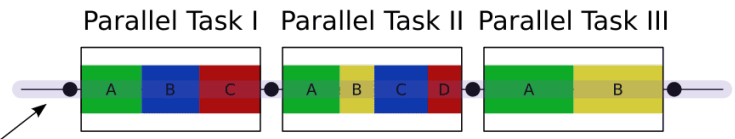
Commonly known as Pthreads.

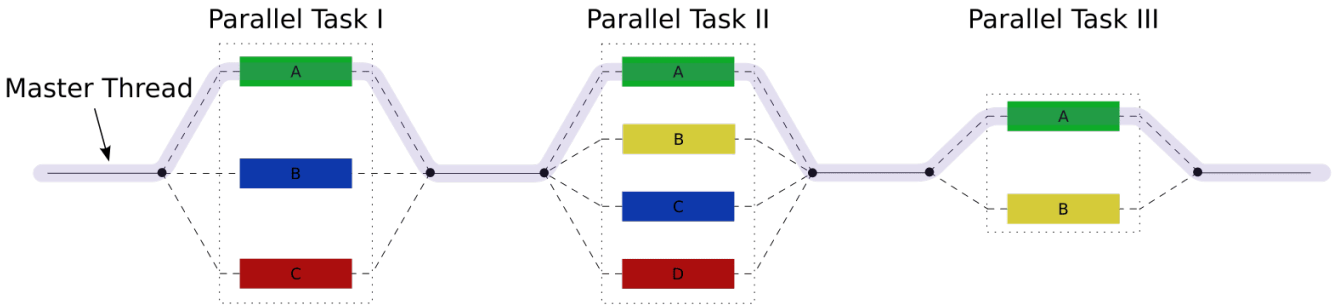
Most hardware vendors offer Pthreads in addition to their proprietary implementations.

Very explicit parallelism, requires a very significant level of detail by the programmer

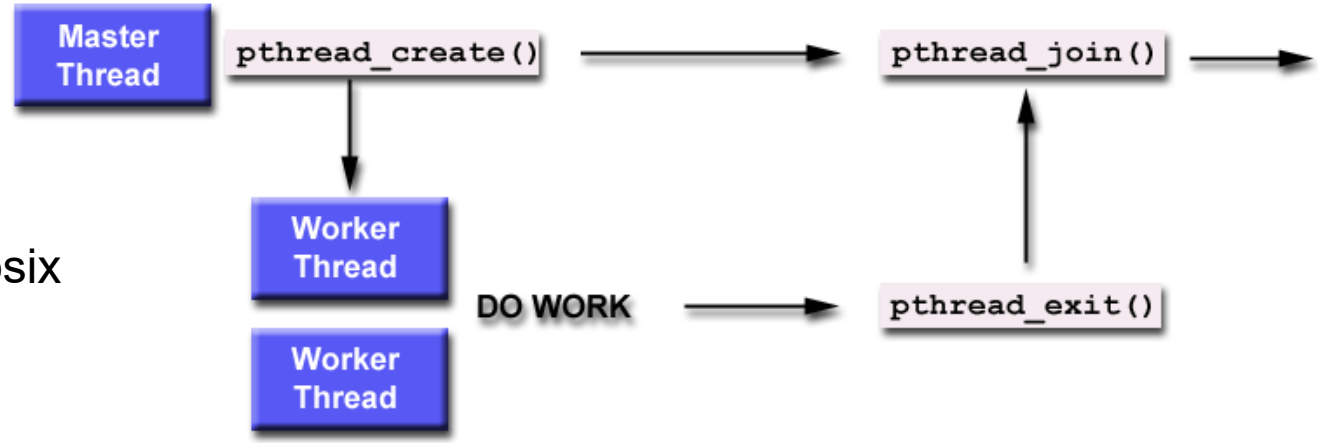
OTHERS … (Qt Threads, Openthreads) … OpenMP!

Threads Model





With Posix



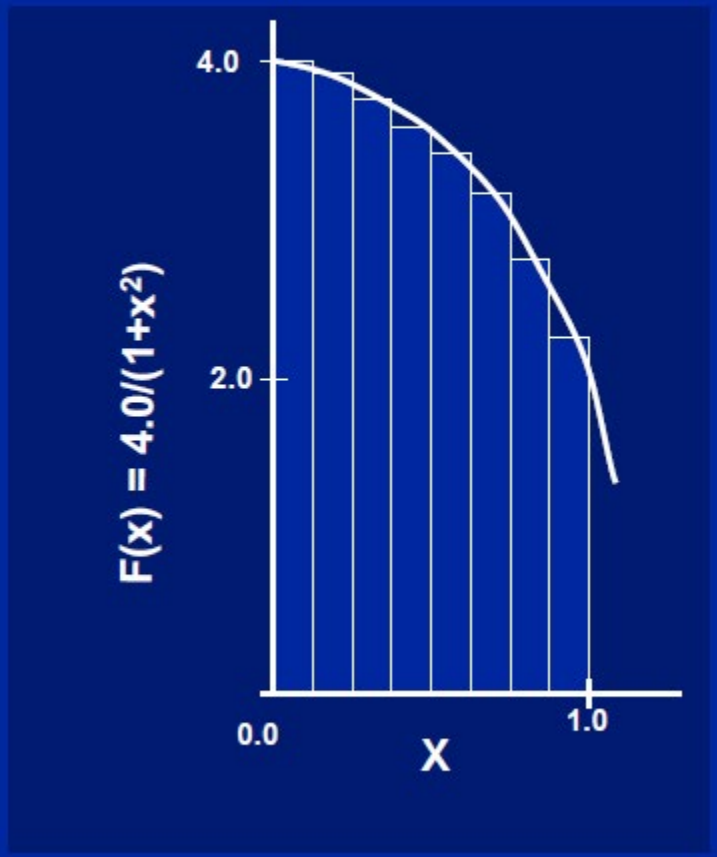
Shared Memory Model

- Creation of static or dynamic processes (fork, exec, joins, etc.)

- Coordination between processes is typically done via 3 primitives:
  - Shared variables
  - Critical section (mutex, semaphores, etc)
  - Synchronization (Barrier)

π computation

Numerical Integration



Mathematically, we know that:

$$\int_0^1 \frac{4.0}{(1+x^2)}\,dx = \pi$$

We can approximate the integral as a sum of rectangles:

$$\sum_{i=0}^{N} F(x_i)\Delta x \approx \pi$$

Where each rectangle has width Δx and height $F(x_i)$ at the middle of interval i.

Serial program

```
static long num_steps = 100000;
double step;
int main ()
{       int i;   double x, pi, sum = 0.0;

        step = 1.0/(double) num_steps;

        for (i=0;i< num_steps; i++){
                x = (i+0.5)*step;
                sum = sum + 4.0/(1.0+x*x);
        }
        pi = step * sum;
}
```

Race Condition (cont.)

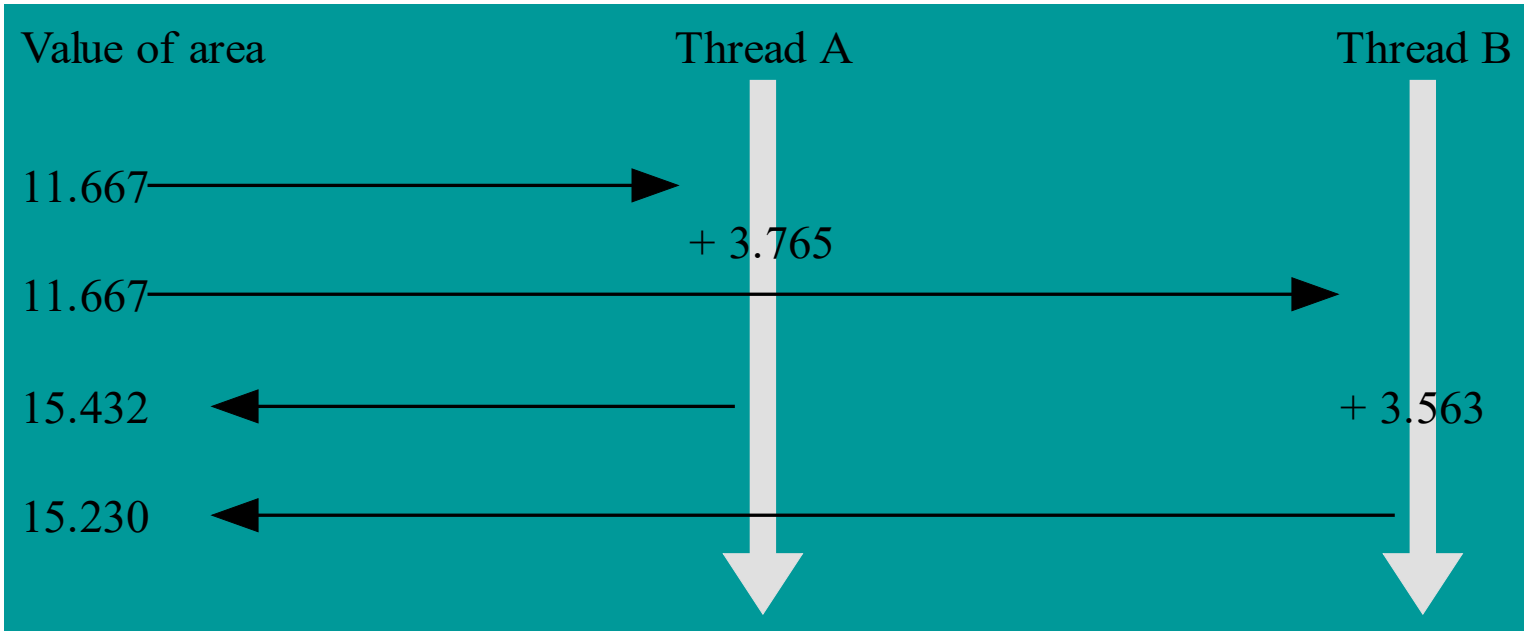
- Let's use two threads A and B to perform in parallel the summation: Thread A computes the first "half" of the interval, thread B the second "half"
- ... we will set up a **race condition** in which one process may "race ahead" of another and not see its change to shared variable **`sum`**

`sum` 15.230 **Answer should be 18.995**

Thread A 15.432 Thread B 15.230

```
sum += 4.0/(1.0 + x*x)
```

Race Condition Time Line



Synchronization

Synchronous communication operations

Involves only those tasks that perform a communication operation

When a task executes a communication, a form of coordination is required with other tasks involved in the communication

For example, before a task can execute a send operation, it must first receive an acknowledgment from the receiving task, which indicates that the "okay to send ..."

Synchronization

Barrier

- Usually involves all tasks
- Each task executes the code until it "reaches" a barrier. Thereafter, it stops or "freezes"
- When the last task reaches the barrier, all tasks are synchronized

Synchronization

```
Do i=1,100
    a(i) = b(i)+c(i)
Enddo
Do i=1,100
    d(i) = 2*a(101-i)
Enddo
```

i=1..25 | 26..50 | 51..75 | 76..100
execute on the 4 processors

BARRIER synchronization

i=1..25 | 26..50 | 51..75 | 76..100
execute on the 4 processors

Synchronization

- is necessary
- may cause
  - **idle time on some processors**
  - **overhead to execute the synchronization primitive**

Synchronization

Lock / semaphore

- Involves any number of tasks
- It is typically used to **serialize** (protect) access to global data or a section of code. Only one task at a time may use (own) the lock / semaphore / flag
- The first task acquires the lock sets it.
- This task can therefore, in safe mode, access data or code
- Other tasks can try to acquire the lock, but must wait until the task that owns it releases it
- It may be **blocking** or **non-blocking**

Another example

```
int private_x;  //local variable
shared int sum = 0;
…
sum = sum + private_x;
```

Es: Code executed on each proc

Example: Possible scenario on 2 processes:

Step	**Thread 0**	**Thread 1**
0	get `sum=0`	end `private_x` computation
1	get `private_x=2`	get `sum=0`
2	sum `0 + 2`	get `private_x=3`
3	save `sum`	sum `0 + 3`
4		save `sum 3`

Sum (`sum`) should be 5 !

Shared Memory Model

- Critical section!
- Binary semaphore!
- The operation `sum = sum + private_x` MUST be performed by a thread at a time!

```
shared int s = 1;
while (!s);                 // wait until s=1
s=0;                        // close access
sum = sum + private_x;      // critical sect.
s = 1;                      // reopen access
```

Shared Memory Model

- **Atomicity**
- While a thread checks `s = 1` to test if it is okay to enter the critical section, another thread might store `s = 0` !
- Two special functions are adopted:

```
void P(int* s);
void V(int* s);
```

- `P` has the effect of preventing other threads from accessing `s` once a process exits the loop
- `V` sets `s` to 1, but ATOMICALLY
- Identified by Dijkstra in 1968. The letter `P` in Dutch stands for **passeren**, which means "go" and the letter `V` comes from the Dutch **vrijgeven**, which means "release"

Shared Memory Model

- Finally, to wait until all the threads have finished before printing, we use a barrier:

```
int private_x;
shared int sum =0;
shared int s = 1;
...
P(&s);
sum = sum + private_x;
V(&s);
Barrier();
if (I am thread 0)
      printf("sum = %d\n", sum);
```

Cache Coherence in Multiprocessor Systems

- Interconnection networks **provide** the basic mechanisms for the transfer of data

- However, in the case of shared address space machines, additional hardware is required to **coordinate access** to the data that may have **multiple copies** in the network

Cache Coherence in Multiprocessor Systems

When the value of a variable changes, all other copies must be **invalidated** or **updated**

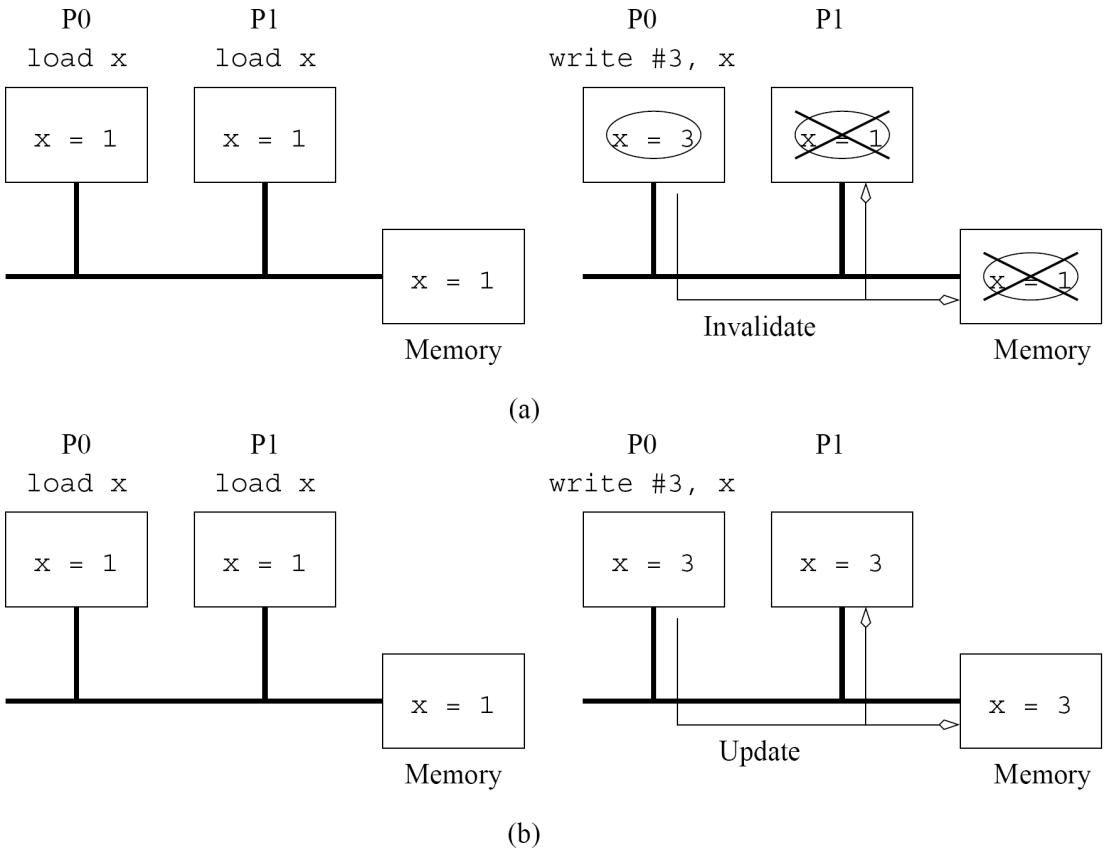


Figure 2.21 Cache coherence in multiprocessor systems: (a) Invalidate protocol; (b) Update protocol for shared variables.

Cache Coherence: Update or Invalidate Protocols

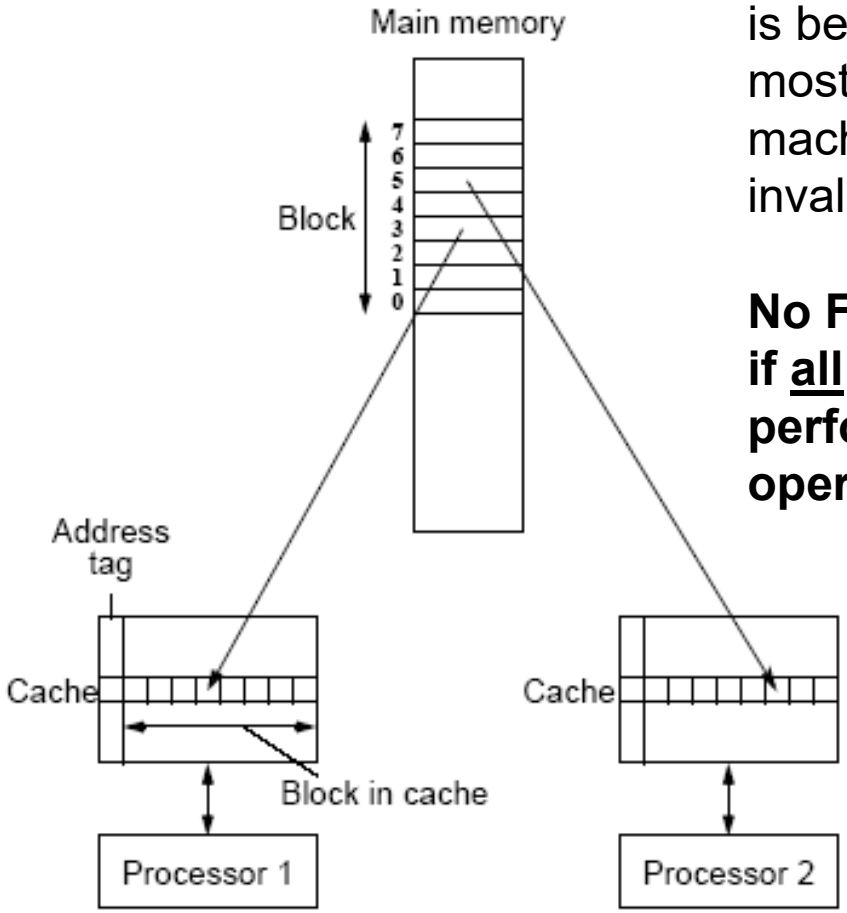
- If a processor reads a value only once, and it is not needed any more, an **update protocol** can generate a **<u>significant overhead</u>**
- If two processors working in "interleaved" access a variable, it is better to use an update protocol
- Both protocols suffer from **<u>false sharing</u>** overheads, a typical performance-degrading usage pattern

<u>Most machines use invalidating protocols</u>

False Sharing

Different parts of a block (**cache line – 32-512 bytes**) required by different processors but **not the same data.**

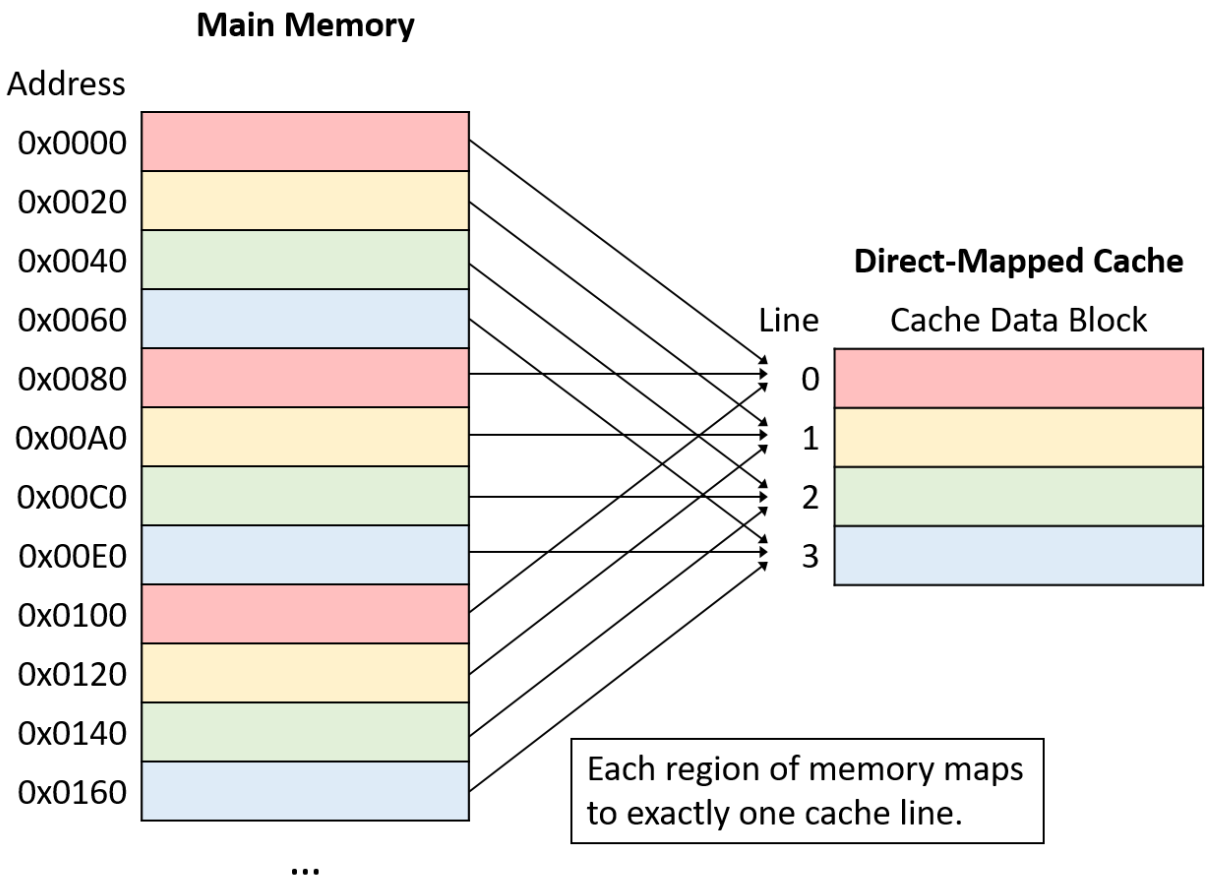
In this case, if a processor writes a part of the block, copies of the complete block in other caches need to be updated or invalidated, **degrading the performance** of the system, given that the actual data (i.e., the entire block) is **not shared**



The use of update protocols is better, although most modern machines adopt invalidate protocols.

No False Sharing if all processors perform READ operations!

Cache line



All of the memory regions that map the same cache line (that is, chunks of the same color in the figure) compete for space in the same cache line, so only one region of each color can reside in the cache at a time.

Example (scalar variables)

```
struct foo {
    int x;
    int y;
};

static struct foo f;

/* The two following functions are running concurrently: */

int sum_a(void)
{
    int s = 0;
    int i;
    for (i = 0; i < 1000000; ++i)
        s += f.x;
    return s;
}

void inc_b(void)
{
    int i;
    for (i = 0; i < 1000000; ++i)
        ++f.y;
}
```

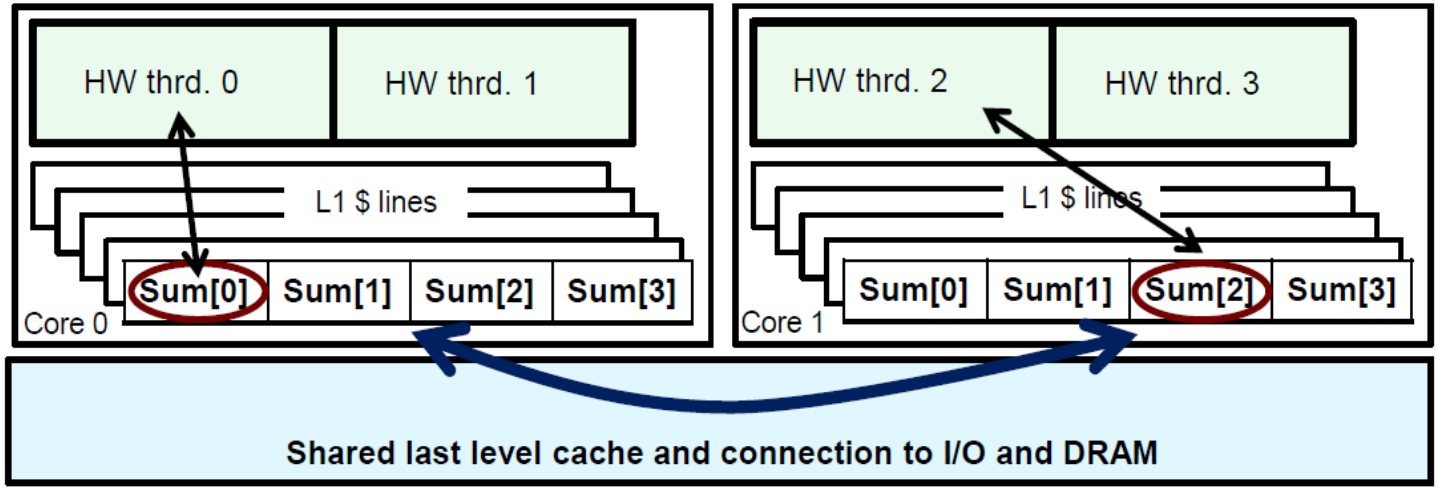
Here, `sum_a` may need to continually re-read `x` from main memory (instead of from cache) even though `inc_b`'s concurrent modification of `y` is irrelevant for `sum_a` !

Vector Sum (shared memory)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

// Vector Sum - False Sharing

#define numThreads 10

int somma;
long num=100000000;
long sum[numThreads];
int slot = num/numThreads;

void* run(void * arg){
    ...

    for (long i = start; i < end; i++)
        sum[id]+=i;

   ...
}

int main(int arg, char* argv[])
{
    pthread_t thid[10];

    for(int i=0; i<numThreads;i++){
      ...
        int ris = pthread_create(&thid[i], NULL, &run, ip);

    }

    long sum=0;
    for(int i=0; i<numThreads;i++){
        pthread_join(thid[i], NULL);
    }
    for (int i = 0; i < numThreads; i++)
        somma+=sum[i];

    printf("sum=%ld\n",somma);
}
```

Why such poor scaling? False sharing!

- If independent data elements happen to sit on the same cache line, each update will cache lines to «slosh back and forth» between threads…. FALSE SHARING.
- **Example:** Vector Sum



- If you promote scalars to an array to support creation of a SPDM program, the array elements are contiguous in memory and hence share cache lines… results in poor scalability
- **Padding solution: Pad arrays so elements you see are on distinct cache lines**

Vector Sum (shared memory)

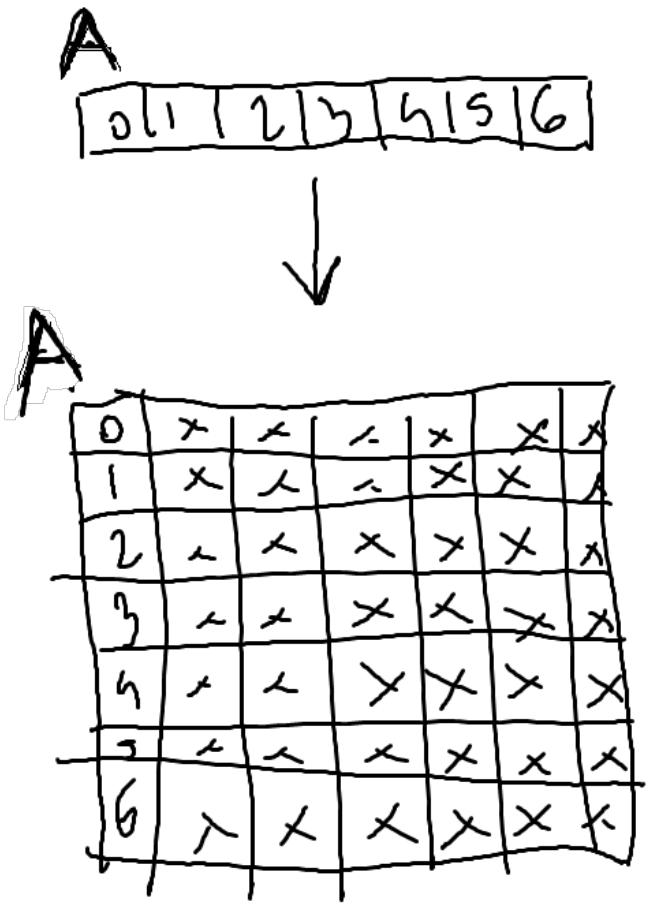
```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

// Vector Sum - False Sharing

#define numThreads 10

int somma;
long num=100000000;
long sum[numThreads];
int slot = num/numThreads;

void* run(void * arg){
    ...

    for (long i = start; i < end; i++)
        sum[id]+=i;

   ...
}

int main(int arg, char* argv[])
{
    pthread_t thid[10];

    for(int i=0; i<numThreads;i++){
      ...
        int ris = pthread_create(&thid[i], NULL, &run, ip);

    }

    long sum=0;
    for(int i=0; i<numThreads;i++){
        pthread_join(thid[i], NULL);
    }
    for (int i = 0; i < numThreads; i++)
        somma+=sum[i];

    printf("sum=%ld\n",somma);
}
```

Padding needed here!

1D array should become 2D array with padding:

`sum[id][0]`

Padding for false sharing mitigation



A[N]

A[N][PAD]

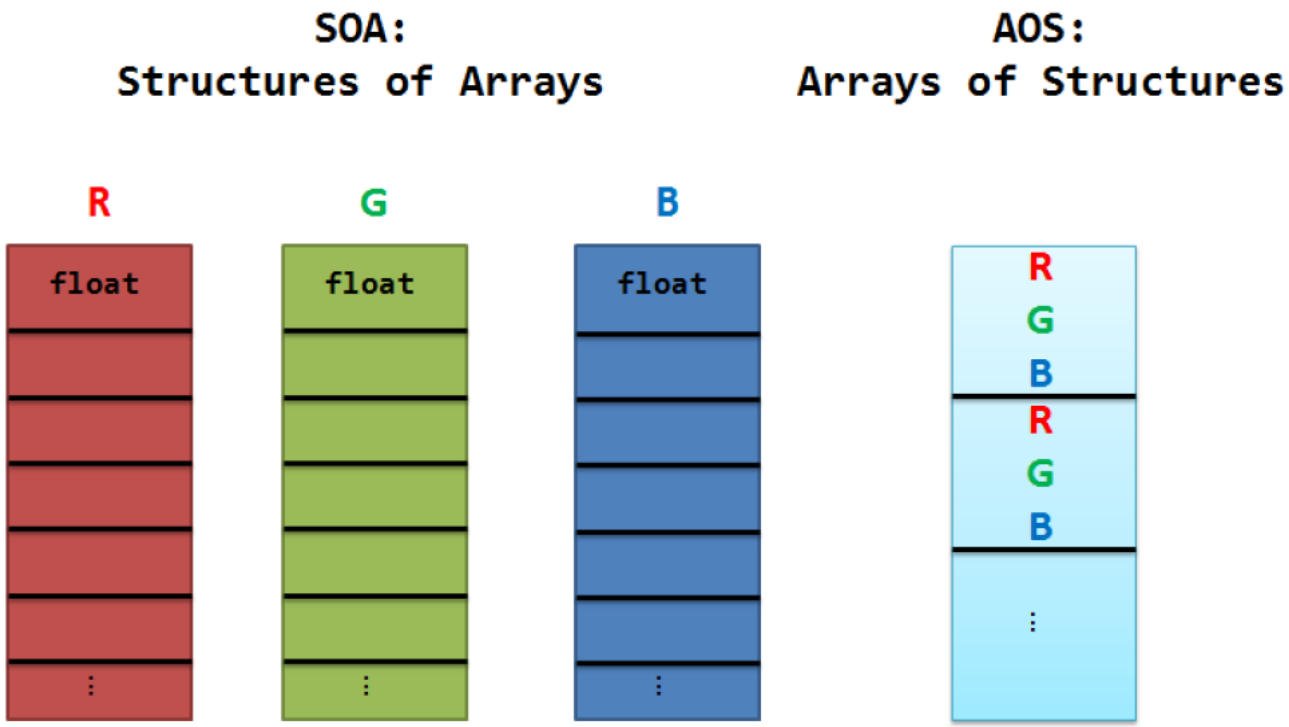
PAD = 32, 64 ... 512?

Example with padding

```
struct foo {
    int x[PADSIZE];  // scalar variable padding -> 1D array
    int y[PADSIZE];
};

static struct foo f;

/* The two following functions are running concurrently: */

int sum_a(void)
{
    int s = 0;
    int i;
    for (i = 0; i < 1000000; ++i)
        s += f.x[0];
    return s;
}

void inc_b(void)
{
    int i;
    for (i = 0; i < 1000000; ++i)
        ++f.y[0];
}
```

Here, `f.x[0]` contains the “real” data to be used, and so `f.y[0]` should be in a distinct cache line (hopefully!)

SOA vs AOS


SOA:
Structures of Arrays
AOS:
Arrays of Structures
R
G
B
float
float
float
R
G
B
R
G
B

Memorizzazione dati in C

- **Problema: come memorizzare dati vettoriali/matriciali**
  - Ogni elemento ha più attributi (es: x, y, z)
  - Due layout principali:

- **Array of Structures (AoS)**

```
struct Point { float x, y, z; };
Point arr[N];
```

- **Structure of Arrays (SoA)**

```
struct Points {
  float x[N], y[N], z[N];
};
```

- **Differenza:**
  - AoS → dati di un elemento sono contigui
  - SoA → dati dello stesso attributo sono contigui

Vantaggi AoS

- **Pro**
- Più intuitivo (modello "oggetto")
- Migliore località se usi *tutti i campi* insieme
- Codice più leggibile e facile da mantenere
- Naturale per OOP e strutture complesse
- 👎 **Contro**
- Cache meno efficiente per accessi parziali
- Difficile vectorization (SIMD)
- Possibile padding → spreco memoria
- 👉 Ideale quando:
- Usi spesso tutti i campi di ogni elemento
- Accesso per oggetto (es: simulazioni semplici)

Vantaggi SoA

- **Pro**
- Migliore cache locality per accessi per campo
- Ottimo per SIMD / vectorization
- Più veloce su grandi dataset numerici
- Migliore bandwidth della memoria
- 👎 **Contro**
- Meno intuitivo
- Codice più complesso
- Accesso a un singolo elemento più costoso
- 👉 Ideale quando:
- Elaborazioni numeriche intensive
- GPU / HPC / data-oriented design
- Loop su un singolo attributo (es: tutte le x)

Best practices

Caso d’uso	Migliore scelta
Accesso per oggetto	AoS
Accesso per campo	SoA
SIMD / vectorization	SoA
Leggibilità	AoS
Performance HPC	SoA

La scelta tra AoS e SoA non è solo di design, ma influisce direttamente su cache, parallelismo e performance!

The Maintenance of Coherence by Invalidating Protocols

- Each copy of the data is associated with a **state**
- The possible states of a variable are: **shared, invalid or dirty**

- In the **shared state**, there are multiple valid copies of the data
- If a processor modifies a value, an invalidation must be generated on other copies of other processors. The copy becomes **dirty**

- In the **invalid state**, the copy of the data is invalid, and a reading generates a data request
- *In the dirty state, only one copy exists, and therefore no invalidation must be generated*

The Maintenance of Coherence by Invalidating Protocols

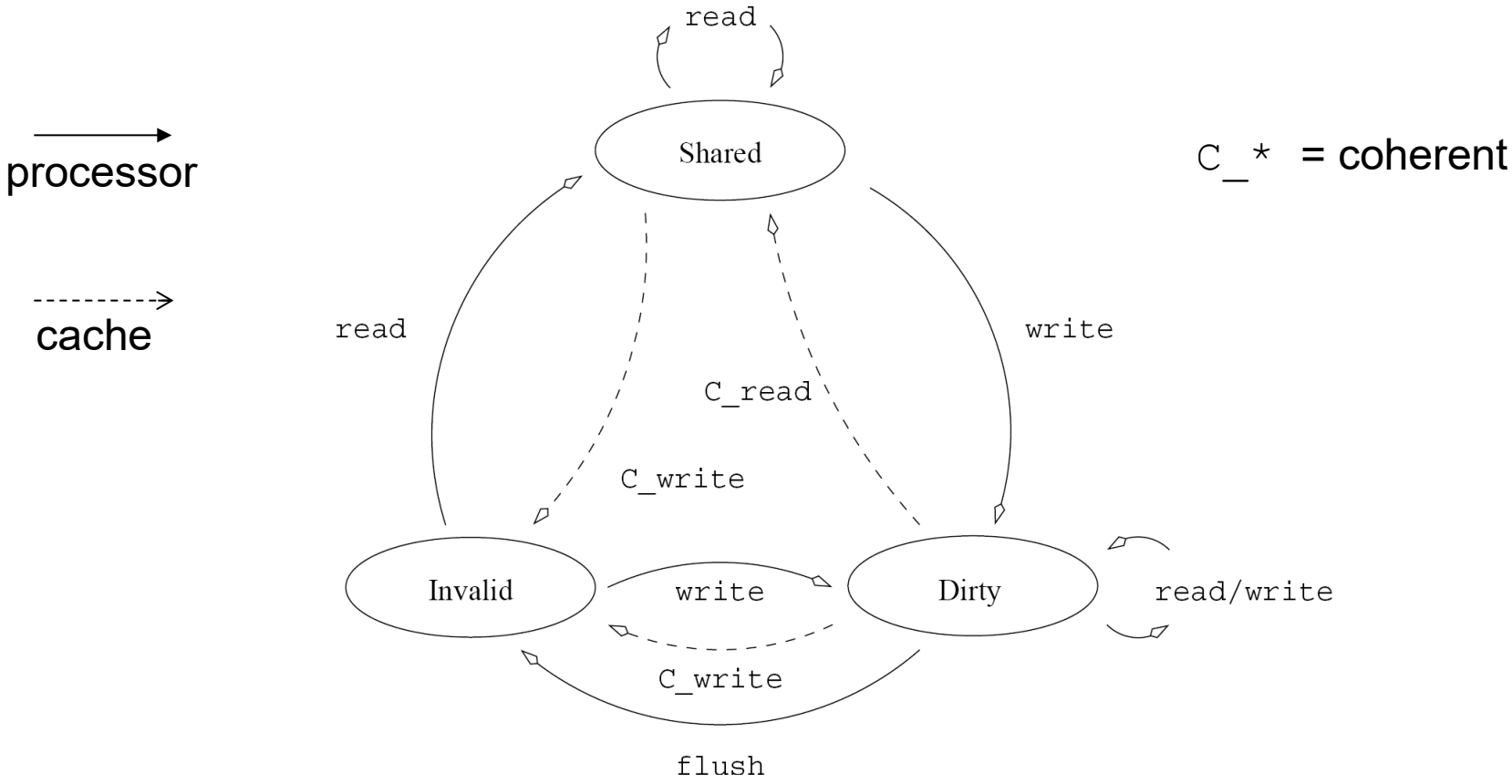


Figure 2.22 State diagram of a simple three-state coherence protocol.

The Maintenance of Coherence

Time ↓

Instruction at Processor 0	Instruction at Processor 1	Variables and their states at Processor 0	Variables and their states at Processor 1	Variables and their states in Global mem.
				x = 5, D y = 12, D
read x		x = 5, S		x = 5, S y = 12, S
	read y		y = 12, S	
x = x + 1		x = 6, D		x = 5, I y = 12, I
	y = y + 1		y = 13, D	
read y		y = 13, S	y = 13, S	y = 13, S
	read x	x = 6, S	x = 6, S	x = 6, S
x = x + y		x = 19, D	x = 6, I	x = 6, I
	y = x + y	y = 13, I	y = 19, D	y = 13, I
x = x + 1		x = 20, D		x = 6, I
	y = y + 1		y = 20, D	y = 13, I

Figure 2.23 Example of parallel program execution with the simple three-state coherence protocol discussed in Section 2.4.6.

Snoopy Cache Systems

How are invalidates sent to the right processor?

In **snoopy caches** (buses, rings), there is a broadcast device that "listens" and invalidates all requests for readings, performing the appropriate operations on the basis of the previous diagram

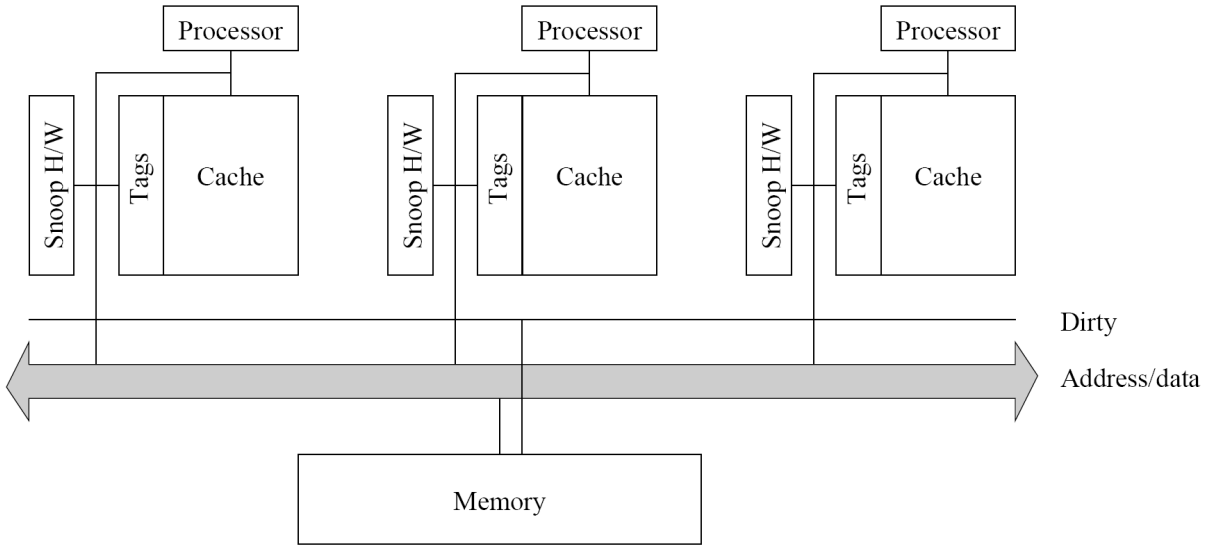
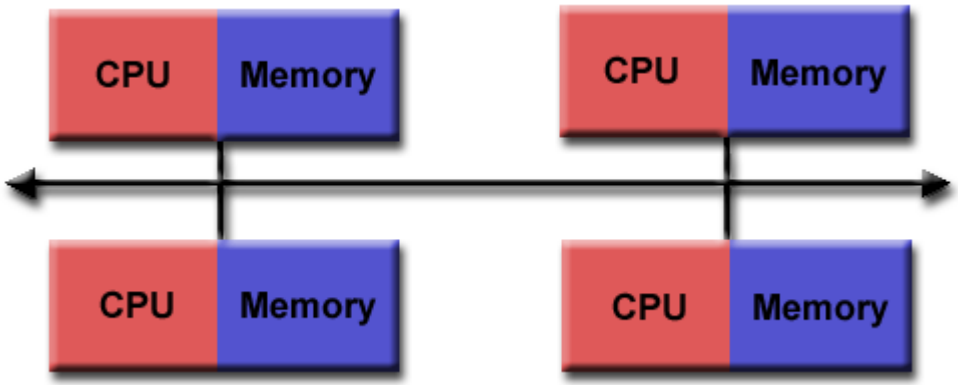


Figure 2.24 A simple snoopy bus based cache coherence system.

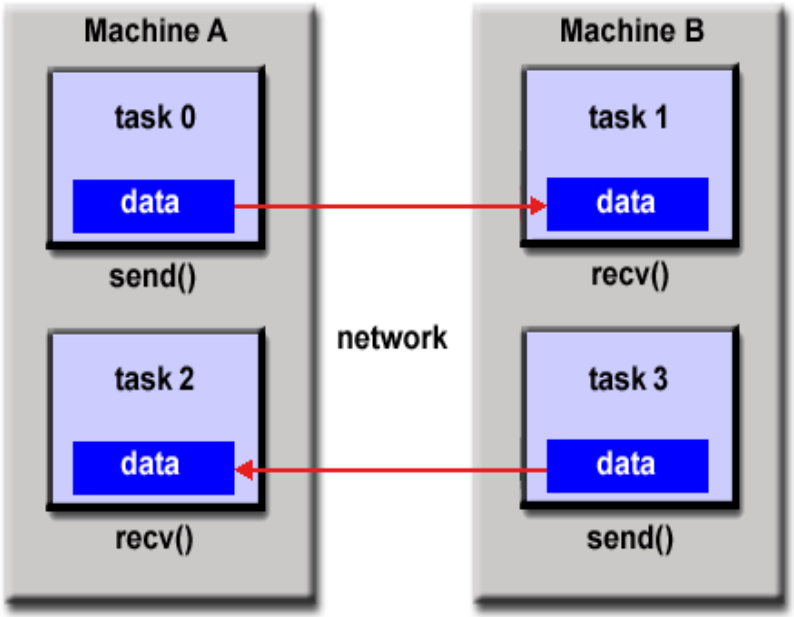
Message-Passing Platforms (Distributed Memory)

- These platforms include a set of processors with their own **exclusive memory** (from the logical point of view)
- Examples are clusters of workstations and multicomputer with space-address-not-shared (non-shared-address-space multicomputers)
- These platforms are programmed using (variants of) **send** and **receive** primitives
- Libraries such as MPI and PVM provide such primitives
- They do not need, obviously, the use of cache-coherence protocols



Message Passing Model

1. A set of tasks that use their memory during the computation. Multiple tasks can reside on the same physical machine and on different machines
2. The tasks exchange data through communications by sending and receiving messages.
3. The data transfer usually requires cooperative operations for each process. For example, a send operation must have a corresponding receive



From a programmer point of view, the message passing implementations usually include a library of functions that are incorporated in the code. The programmer is responsible for determining the parallelism.

MPI (Message Passing Interface), standard de-facto for the Message Passing model

Message Passing Model

- Single Program Multiple Data (SPMD) Approach

```
if (my_process_rank ==0)
   MPI_Send(&x, 1, MPI_FLOAT, 1, 0, MPI_COMM_WORLD);
else
   if (my_process_rank ==1)
     MPI_Recv(&x, 1, MPI_FLOAT, 0, 0, MPI_COMM_WORLD,
       &status);
...
if (my_process_rank ==3)
   MPI_Send(&x, 1, MPI_FLOAT, 2, 0, MPI_COMM_WORLD);
else
  if (my_process_rank ==2)
     MPI_Recv(&x, 1, MPI_FLOAT, 3, 0, MPI_COMM_WORLD,
       &status);
```

The MPI paradigm

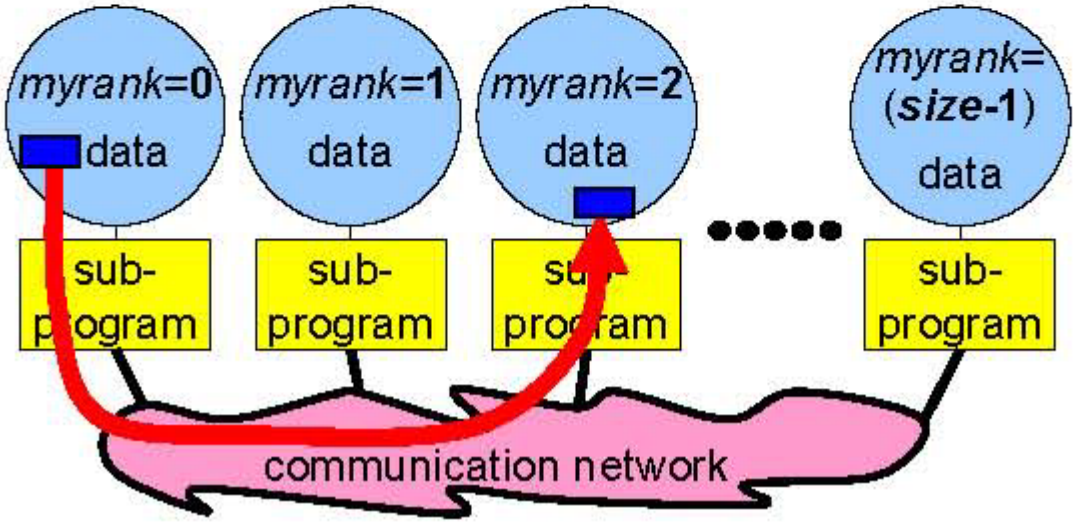
Each processor in a message passing program runs a ***sub-program***

- written in a conventional sequential language, e.g., C or Fortran,
- typically the same on each processor (SPMD)

All work and data distribution is based on value of ***myrank***

- returned by special library routine

Communication via special send & receive routines (***message passing***)



Communication

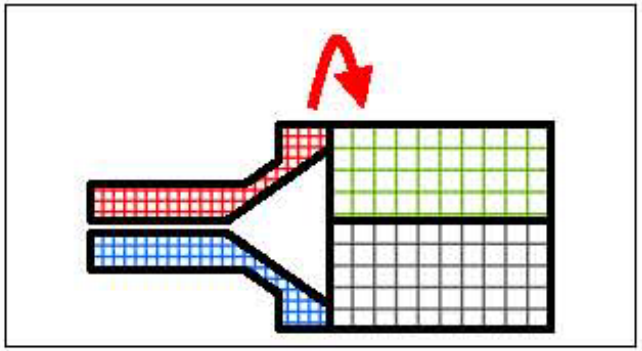
```
Do i=2,99
    b(i) = a(i) + f*(a(i-1)+a(i+1)-2*a(i))
Enddo
```

Communication is necessary on the boundaries

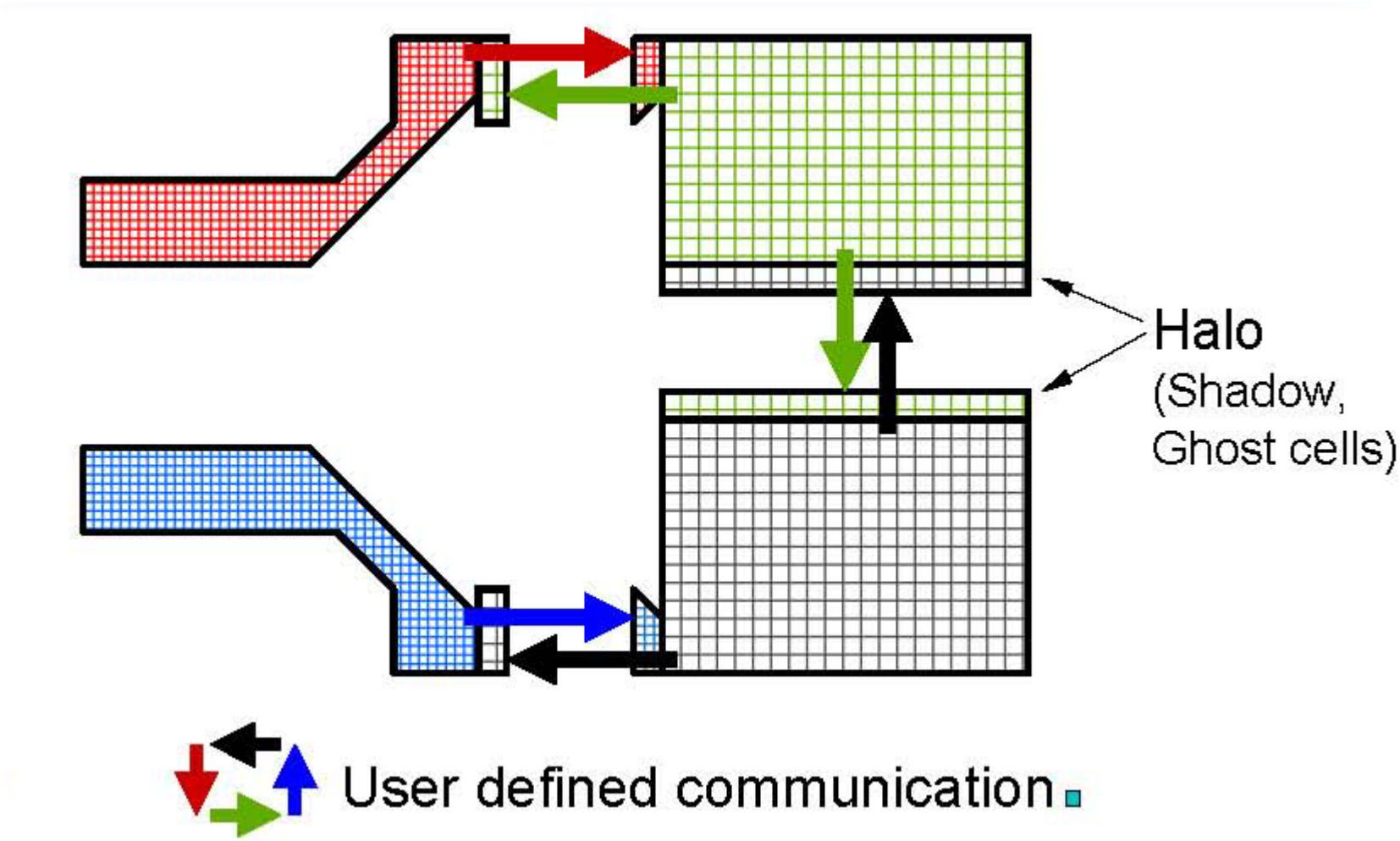
- e.g. b(26) = a(26) + f*(a(25)+a(27)-2*a(26))

a(1:25),	b(1:25)
a(26,50),	b(51,50)
a(51,75),	b(51,75)
a(76,100),	b(76,100)

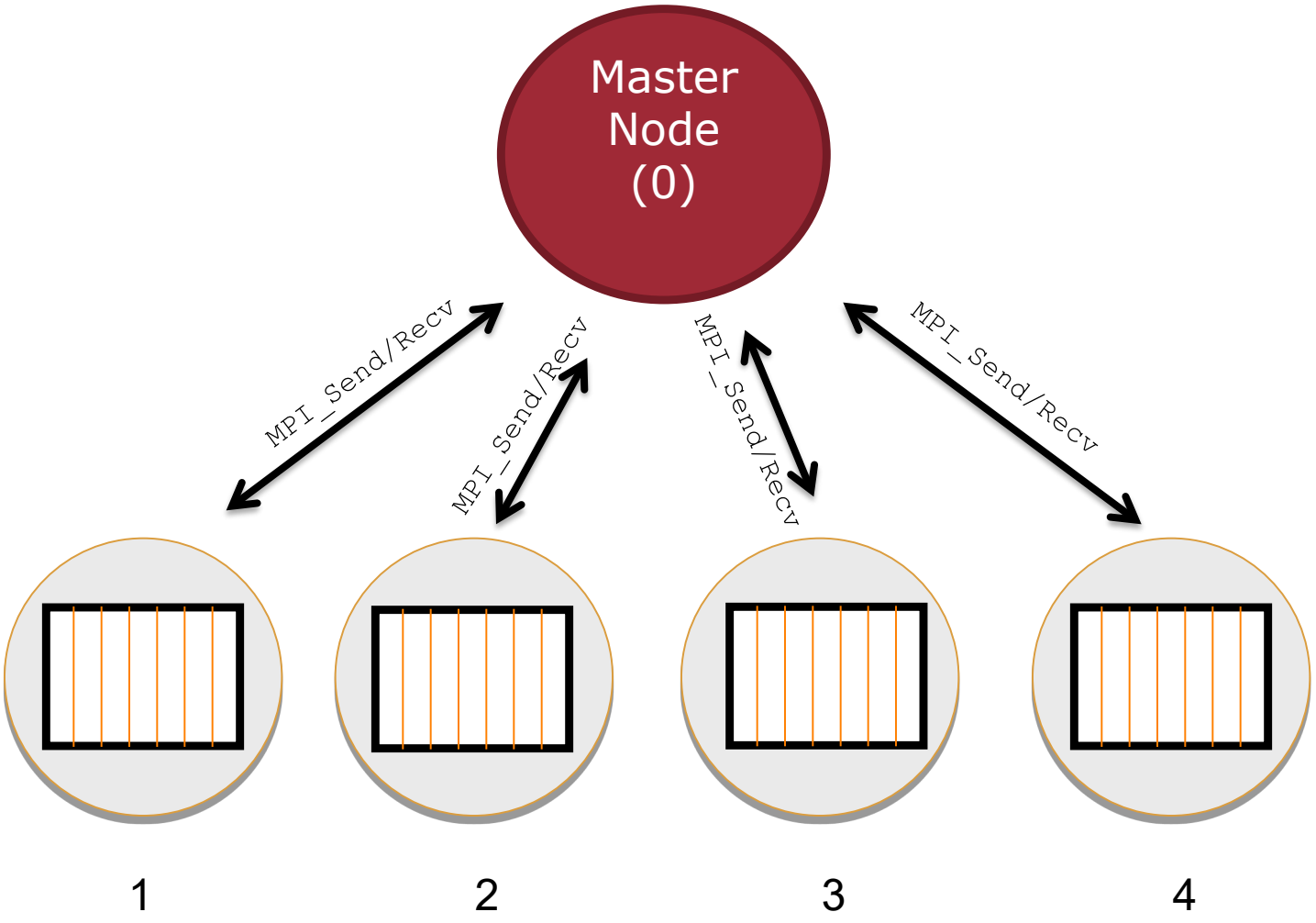
- e.g. at domain boundaries



Ghost/Halo cells



Typical Master-Slave pattern



Vector sum (dynamic allocation)

```
#include <mpi.h>
#include <stdio.h>

#define MAXSIZE 100000000

int main(int argc, char** argv)
{
    double* localdata = NULL;
    int i, x, low, high;
    int myid, numprocs;
    int dest, source;
    double myresult, result, result_temp;
    double starttime, endtime;
    MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &myid);

    x = MAXSIZE/numprocs;
    // alloco solo quello che mi serve, non TUTTO MAXSIZE!
    localdata = new double[x];

    result = 0;
    myresult = 0;

    // Init...(each process, even master)
    // Oppure alloco data solo su MASTER e poi spezzetto sui procs...
    for (i=0; i<x;i++)
        localdata[i] = myid;

    MPI_Barrier(MPI_COMM_WORLD);
    starttime = MPI_Wtime();


    // Compute my result (even Process 0 will do it - MS democratico)
    for (i=0; i<x; i++)
        myresult = myresult + localdata[i];

     if (myid == 0){
        result = myresult;
        for (source=1; source<numprocs; source++){
            MPI_Recv(&myresult, 1, MPI_DOUBLE, MPI_ANY_SOURCE, 0, MPI_COMM_WORLD, &status);
        result = result + myresult;
        }
     }
        else
            MPI_Send(&myresult, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD);

    MPI_Barrier(MPI_COMM_WORLD);
    endtime = MPI_Wtime();

    if (myid == 0){
        printf("Sum is %e.\n", result);
        printf("Elapsed time: %f\n", 1000*(endtime - starttime));
    }
    delete[] localdata;
    MPI_Finalize();
    exit(0);
```

Master

Slave

Pros & Cons (Message Passing)

Advantages:

Memory is **scalable** with the number of processors. If you increase the number of processors, memory size is increased proportionally.

Each processor can rapidly access its memory without overhead caused to cache coherency maintenance

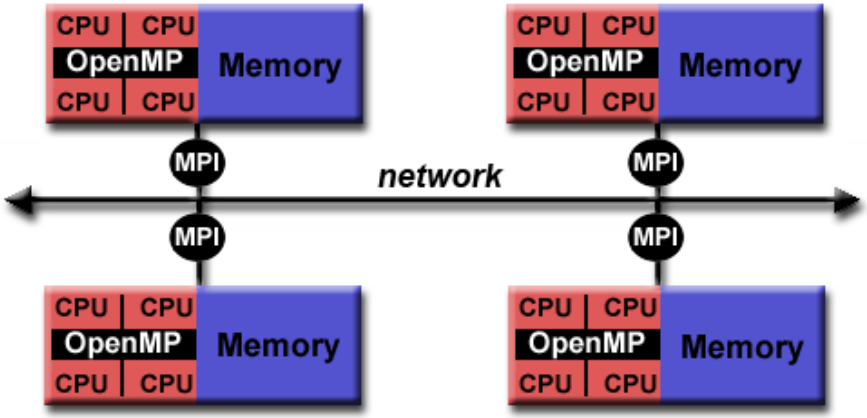
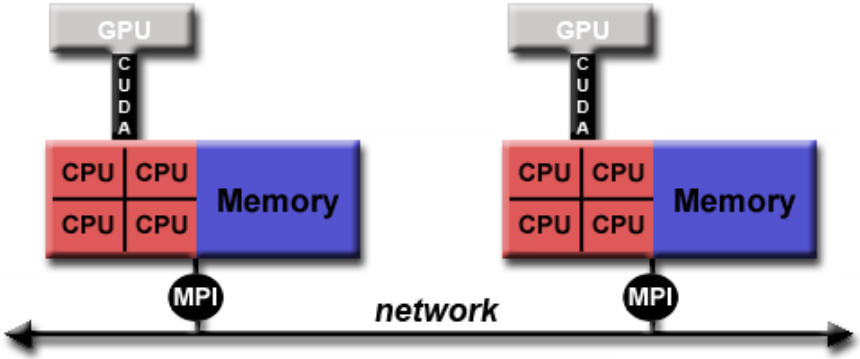
Actual cost: You can use common "off-the-shelf" processors and networks

Disadvantages:

The **programmer is responsible** for most of the details associated with the communication of data between processors

It may be difficult to map existing data structures, due to the global memory in this type of organization.

Non-uniform access times (NUMA)

Hybrid architectures

- **The most powerful computer architectures employ both shared and distributed memory**
- The shared memory component is usually a cache coherent SMP and so processors of a given SMP access a global address memory
- The distributed memory component is the network of SMP
- The current trend is the use of this kind of machines
- **Advantages and disadvantages**: what is common to the various components taken individually
- Joint use MPI – Pthreads - OpenMP (and GPGPU) !